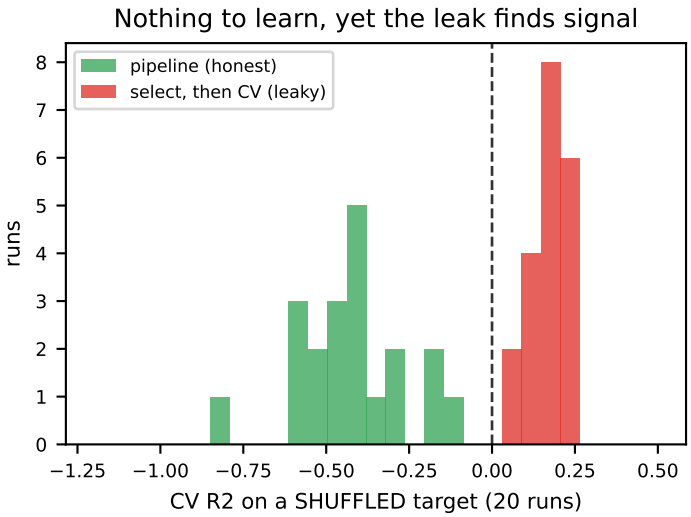
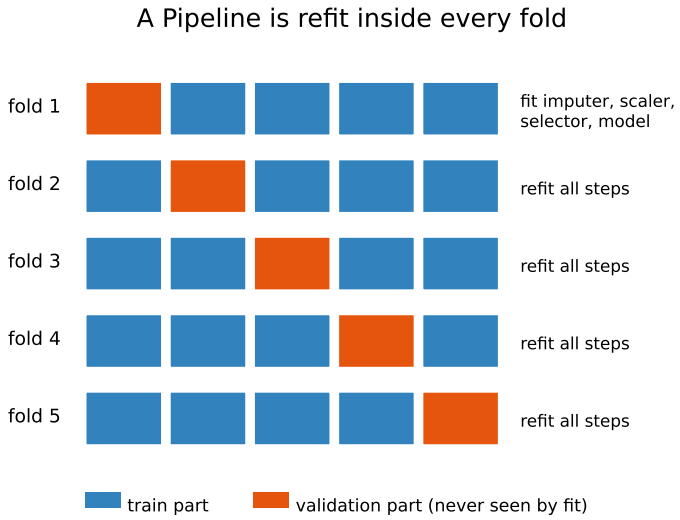
# Day 27 Cheat Sheet: Data Leakage and Pipelines in CV

Week 4: if information from validation rows reaches fit, your score is a lie

| LEAK TYPE | WHAT HAPPENS | FIX |
|---|---|---|
| Preprocessing leak | scaler/imputer/selector fit on ALL rows before CV | put every step in a Pipeline |
| Selection leak | pick top features using every row's target | SelectKBest INSIDE the Pipeline |
| Target leak | a column only known after the outcome | ask: known at prediction time? |
| Group leak | same patient in train and validation folds | GroupKFold(groups=patient_id) |
| Time leak | future rows used to predict the past | TimeSeriesSplit / split by date |
| Test peeking | tuning or choosing on the test set | CV on TRAIN, test scored once |
| SANITY TEST | shuffle y: an honest CV score should be near 0 R2 | if R2 > 0 on noise, hunt the leak |




THIS DATA: 2,400 simulated encounters from 800 patients (3 stays each, seed 27), 8 clinical features,
300 pure-noise markers, albumin 12% missing, total_charges billed after discharge. Split by patient: train 1,800, test 600.
Shuffled target (180 rows, 300 markers): select-then-CV R2 +0.077; pipeline R2 -0.394. 20 shuffles: leaky mean +0.17, honest -0.42.
Target leak: CV MAE 0.439 days with total_charges vs 1.478 with admission-time features.
Group leak (random forest): KFold 1.514 vs GroupKFold 1.578 days. Impute-before-CV: 1.4771 vs 1.4776 (tiny here).
Champion by GroupKFold CV: ridge, clinical. Test once: MAE 1.461 days, RMSE 1.853, R2 0.746.

RULES OF THUMB

- Anything that learns from data (mean, median, scale, selected columns, encodings) must be fit on the training fold only.
- A score that looks too good usually is. Check each feature's timestamp against the moment you predict.
- Hospitals see the same patient again. Split and cross-validate by patient, not by row.
- Some leaks are huge (target), some are small (imputer). The habit matters more than any one number.

```
pipe = make_pipeline(SimpleImputer(strategy="median"), StandardScaler(),
                     SelectKBest(f_regression, k=10), Ridge(1.0))
cross_val_score(pipe, X, y, groups=patient_id, cv=GroupKFold(5),
                scoring="neg_mean_absolute_error")
```

# Infographic: Four ways a score lies

Same hospital data, leaky versus honest evaluation

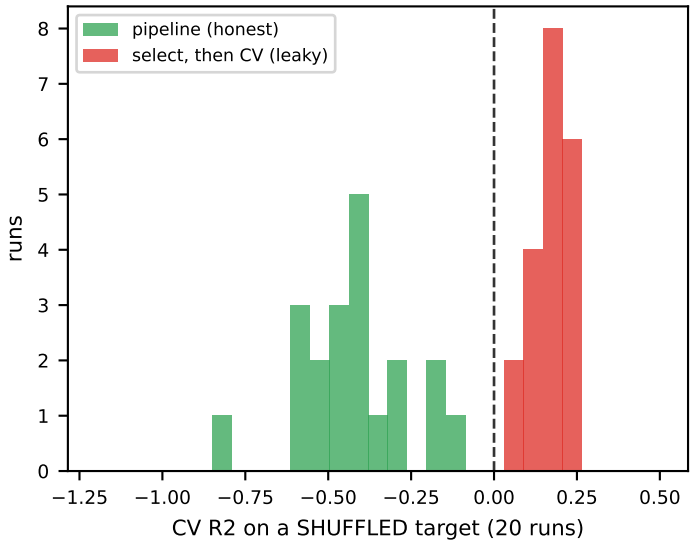
**LEAKY R2 ON NOISE**
**+0.077**

**TARGET-LEAK MAE**
**0.44 d**

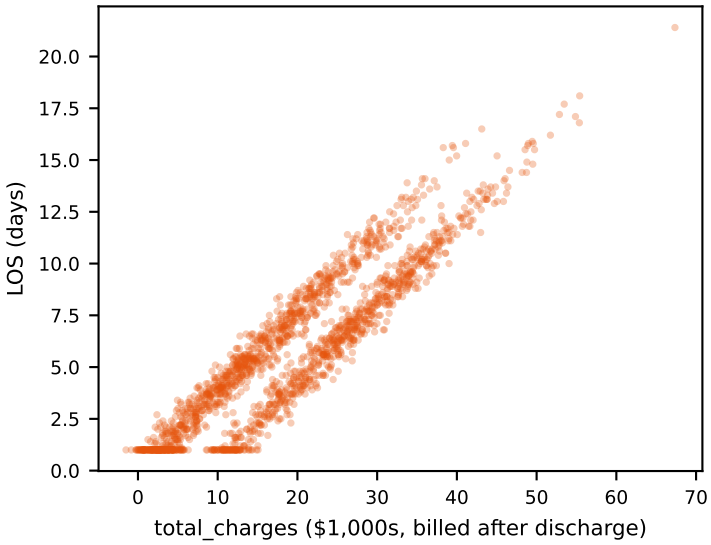
**HONEST CV MAE**
**1.48 d**

**TEST MAE**
**1.46 d**

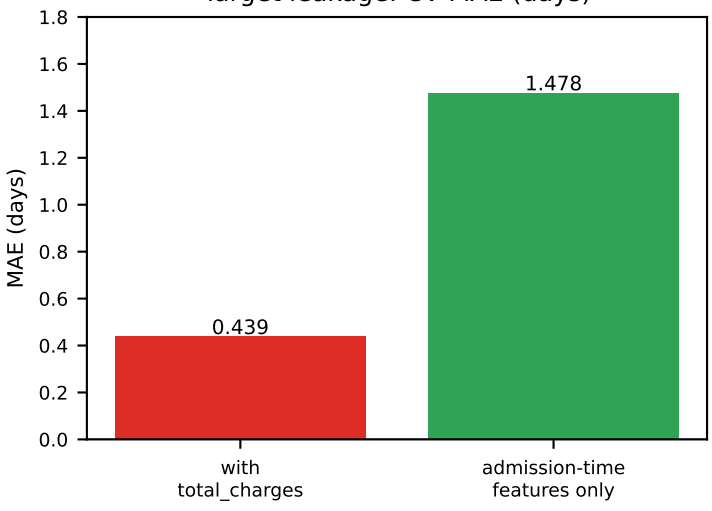
Nothing to learn, yet the leak finds signal



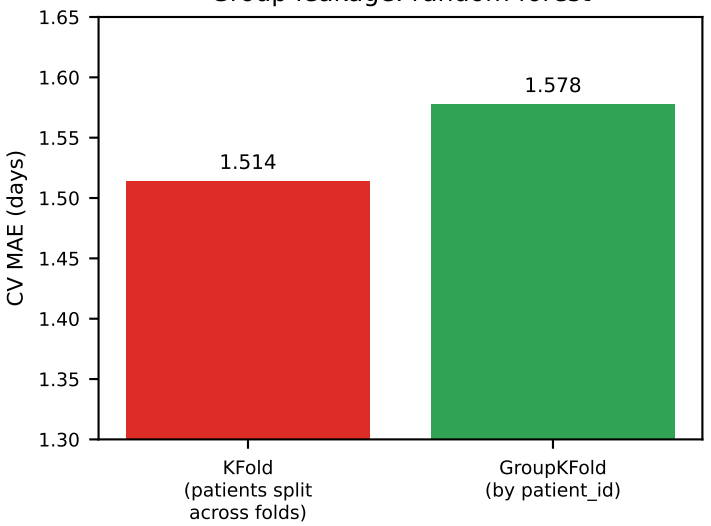

Target leak: r = 0.91



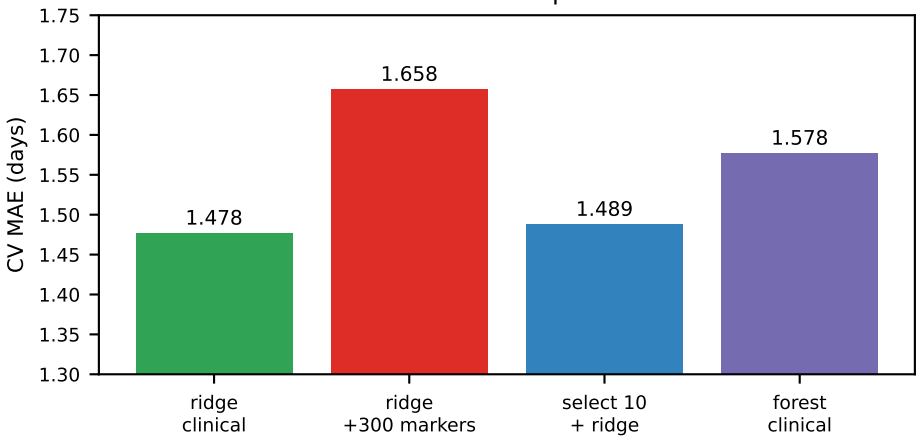

Target leakage: CV MAE (days)




Group leakage: random forest




Honest scorecard: GroupKFold on TRAIN




How to read it

1. On pure noise the leaky setup still reports R2 > 0.
2. total_charges cuts MAE from 1.48 to 0.44, but it does not exist at admission.
3. Splitting a patient's stays across folds flatters the forest by 0.064 days.
4. Champion: ridge, clinical.

# Hands-On Worksheet: ds-genai-90-days / day27

60-90 minutes | notebook: day27/leakage_los.ipynb + day27/README.md

[ ] mkdir day27, conda activate ds90, open day27/leakage_los.ipynb

[ ] Simulate 2,400 encounters from 800 patients (snippet 1, seed 27)

[ ] Note the traps: 300 noise markers, missing albumin, total_charges

[ ] Shuffle the target; run select-then-CV and the Pipeline (snippet 2)

[ ] Explain in a markdown cell why R2 > 0 on noise is impossible honestly

[ ] Split by patient with GroupShuffleSplit; build GroupKFold(5) (snippet 3)

[ ] Compare CV MAE with and without total_charges; label it target leak

[ ] Write a 'known at admission?' column audit for every feature

[ ] Random forest: KFold vs GroupKFold (snippet 4); record the gap

[ ] Impute before CV vs inside the Pipeline; note how small it is

[ ] Build the four-candidate honest scorecard (snippet 5)

[ ] Pick the champion by CV only; score the patient-level test set ONCE

[ ] README: four leaks, charts, champion, limits (simulated data)

[ ] git add day27 && git commit && git push

[ ] Stretch: Snowflake SQL leakage audit by feature timestamp (snippet 7)

```
gss = GroupShuffleSplit(n_splits=1, test_size=0.25, random_state=27)
tr_idx, te_idx = next(gss.split(df, groups=df.patient_id))
pipe = make_pipeline(SimpleImputer(strategy="median"), StandardScaler(), Ridge(1.0))
cross_val_score(pipe, train[clinical], train.los_days,
    groups=train.patient_id, cv=GroupKFold(5), scoring="neg_mean_absolute_error")
```

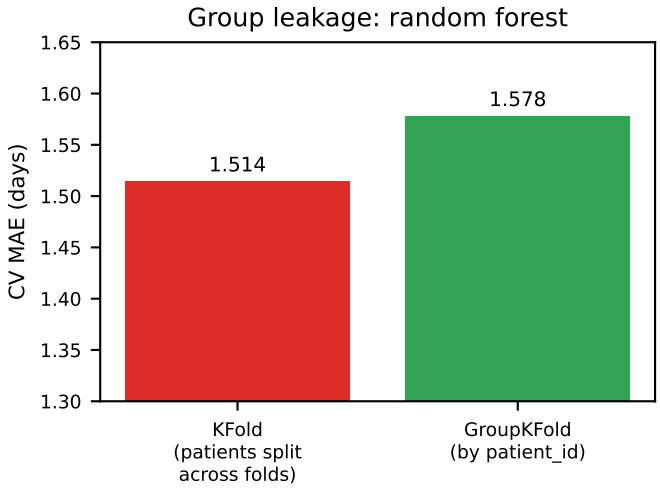
Shuffled R2  leaky: _____  pipeline: _____

CV MAE  with charges: _____  admission only: _____

Forest  KFold: _____  GroupKFold: _____

Champion: _____  Test MAE: _____  R2: _____

Notes: _____



# Day 27 Self-Check Quiz

1. You scaled X with StandardScaler().fit(X) and then ran cross_val_score. What leaked?

_____

2. A readmission model's top feature is discharge_disposition_code. Should you worry? Why?

_____

3. Your CV R2 is 0.08 on a shuffled target. What does that tell you?

_____

4. Why does SelectKBest belong inside the Pipeline rather than before CV?

_____

5. A patient has 3 stays. Why can KFold flatter a random forest here?

_____

6. Which scikit-learn splitter keeps all of a patient's rows together?

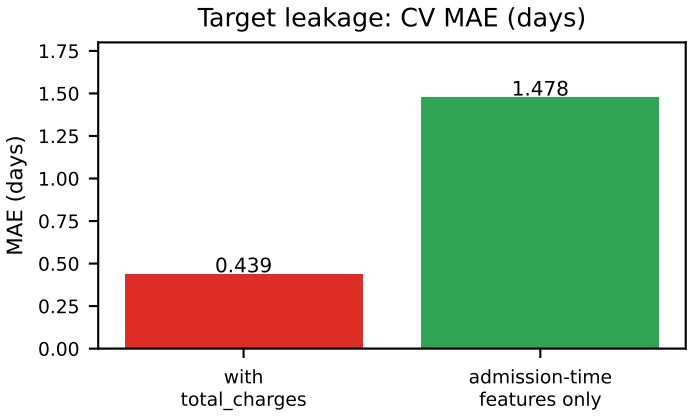
_____

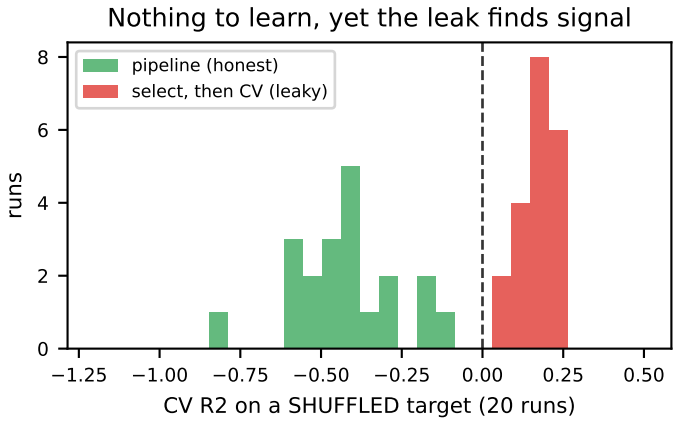
7. Name one leak that was big and one that was tiny in today's data.

_____

8. Where is the only safe place to tune, and when do you score the test set?

_____




Answers:
1 the validation folds' means and SDs reached fit; wrap the scaler in a Pipeline
2 yes: it is set at discharge, after the outcome window starts; check every feature's timestamp
3 there is a leak: with nothing to learn, an honest R2 sits near or below 0 (pipeline gave -0.394)
4 selection uses the target; done before CV it sees validation rows' answers (leaky R2 +0.077)
5 the forest memorizes that patient's other stays: KFold 1.514 vs GroupKFold 1.578 days
6 GroupKFold (and GroupShuffleSplit for the test split), with groups=patient_id
7 big: total_charges (0.44 vs 1.48 days); tiny: imputing before CV (1.4771 vs 1.4776)
8 cross-validation inside TRAIN; the test set is scored once, at the end